



Goblin++

Alpha.12 Day-One Tutorial

Install it. Write it. Run it. Verify it.

A pragmatic scientific language built on Rust

Malin Hess

Version 0.1.0-alpha.12 | 24 September 2026

About this tutorial

This guide is designed to get a new user from a downloaded Goblin++ archive to a useful, verifiable scientific program on the first day. It teaches the language that **exists in alpha.12**. Planned features are identified as future work rather than being presented as available syntax.

The shortest useful definition

Goblin++ is an evidence-first scientific programming language. It combines ordinary programming features, dimension-aware quantities, native FITS access, controlled file output, and a custody workflow that preserves what ran and what it produced.

Who this is for

- Scientists and students who want readable programs with units and built-in evidence handling.
- New programmers who want a compact path through variables, conditions, loops, arrays, functions, and files.
- Experienced programmers evaluating Goblin++ as an auditable scientific workflow language.
- Researchers working with FITS images or binary tables who want explicit, hash-linked data access.

How to use the guide

- 1 Complete Part I to install the command and editor support.
- 2 Type the programs in Part II rather than only reading them.
- 3 Use Part III when you are ready for FITS inputs and generated artifacts.
- 4 Use Part IV before treating a result as evidence or revising frozen work.
- 5 Keep the reference pages nearby while writing your own programs.

Alpha status

Alpha.12 is tested and useful, but it is not a stable-language claim. Receipt schema migration, authenticated authorship, broader FITS semantics, cross-platform release reproduction, compiled FITS/output calls, and modules remain open work.

Licensing and attribution

Copyright © 2026 Malin Hess. Original tutorial prose and diagrams are offered under CC BY 4.0. Code examples are MIT-licensed. The Goblin++ logo is separate project branding. Dependency crates retain their own licenses. See the repository license files for exact scope.

Contents

About this tutorial	2
Who this is for	2
How to use the guide	2
Licensing and attribution	2
Contents	3
Part I - Meet Goblin++	6
1. What problem does Goblin++ solve?	6
2. A brief development history	6
Part II - Install and orient yourself	7
3. Install the macOS arm64 local bundle	7
4. Build from source	7
5. Install the VS Code extension	7
6. Create a workspace	8
Part III - Your first programs	9
7. The smallest useful program	9
8. Names, assignments, comments, and strings	9
9. Values and types	10
10. Units and dimension checking	10
11. Registered constants	11
12. Operators and precedence	11
Part IV - Everyday language features	12
13. Printing and formatting	12
14. GO_PARANOID and seal	12

15. Conditions	12
16. Loops	13
17. Arrays and slices	13
18. Text and g_strings	13
19. Input and program arguments	14
20. G funk: user-defined functions	14
21. Interpretation and compilation	14
Part V - Scientific data and artifacts	15
22. Inspect a FITS file before analysis	15
23. Read a FITS image	15
24. Read a FITS binary table	15
25. Explicit filtered and weighted statistics	16
26. Write text, CSV, TSV, and JSON	16
27. Create deterministic plots	16
Part VI - Evidence, custody, and revision	18
28. What a run preserves	18
29. Freeze, run, verify	18
30. Create a revision and inspect lineage	18
31. Compare runs and audit project state	18
32. Inline Rust: powerful and deliberately difficult	19
Part VII - Guided day-one exercises	20
33. Exercise A: an acceptance report	20
34. Exercise B: inspect the bundled FITS fixture	20
35. Exercise C: compile an everyday program	21

Part VIII - Troubleshooting and scientific habits	22
36. Common errors	22
37. A debugging sequence that preserves meaning	22
38. Scientific review checklist	22
Part IX - Quick reference	24
39. Language keywords and directives	24
40. Everyday built-ins	24
41. FITS built-ins	25
42. Output built-ins	25
43. CLI command card	26
44. Current alpha.12 limits	26
45. Glossary	27
46. Where to go next	27

Part I - Meet Goblin++

1. What problem does Goblin++ solve?

Scientific work often begins with a small calculation and ends with a web of scripts, copied data, unlabeled plots, and results that are difficult to reconstruct. Goblin++ treats execution and evidence as one workflow. A normal run preserves the starting source, stdout, stderr, a receipt, relevant imported data evidence, generated files, and explicitly sealed values.

That does not make a scientific conclusion true. It makes the computational claim inspectable: which source bytes ran, which inputs were accessed, what engine executed them, which outputs were created, and whether those preserved bytes still match their hashes.

Goal	Goblin++ response
Readable everyday code	Variables, text, arrays, functions, branches, loops, input, and command-line arguments.
Scientific correctness aids	Physical dimensions, registered constants, explicit FITS functions, refusal of NaN/infinity, and bounded operations.
Reproducible evidence	Unique run directories, SHA-256 receipts, verification, freezing, revision lineage, semantic diff, and custody ledger.
Native performance path	Rust interpreter by default; explicit native compilation for supported language features.

Evidence is not truth

A perfectly verified run can still encode a poor model, the wrong selection, or a false assumption. Goblin++ helps preserve the computation. Scientific validation still requires independent reasoning, data checks, and domain review.

2. A brief development history

Goblin++ began as a Python reference implementation focused on auditable scientific calculations. The early 0.0.x work established canonical program hashing, run receipts, verification, semantic diff, exact-byte freezing, revision lineage, a checksum custody ledger, normative language material, tutorials, and VS Code editing support.

The 0.1.0 alpha line moved the engine onto Rust while preserving the evidence-first design and adding everyday language features.

Stage	Milestone
Python 0.0.x	Audit receipts, verify/diff, freeze and revision lineage, custody tools, language references, tutorial, editor support.
Rust alpha.1-alpha.3	Native interpreter/compiler foundation, exact-hash inline Rust, native FITS access, audited files and plots.
alpha.4-alpha.6	Ranges, while loops, Boolean comparisons, if/else, switch/case, optional everyday vs paranoid use.
alpha.7-alpha.8	Interactive input, argc/argv, one-dimensional copy-value arrays and slices.
alpha.9-alpha.10	User-defined g_func functions and practical string operations.
alpha.11	Bounded filtered/weighted FITS table summary with explicit selection semantics.
alpha.12	and/or/not, break/continue, direct array iteration, checked remainder, and parse_integer.

The Python 0.0.7 reference remains the normative compatibility anchor while the Rust porting gates are completed. Alpha.12 is a candid development release, not a claim that every old or planned feature has migrated.

Part II - Install and orient yourself

3. Install the macOS arm64 local bundle

The reviewed local bundle contains a macOS arm64 executable and the complete source. Extract it in Finder or with the command line, open Terminal in the extracted folder, and run:

Terminal

```
./install.sh --prefix "$HOME/.local"
rehash
goblin++ --version
```

Expected output includes `goblin++ 0.1.0-alpha.12`. The installer places the command at `~/.local/bin/goblin++`.

If the command is not found

Add `$HOME/.local/bin` to your shell PATH. For zsh, add `export PATH="$HOME/.local/bin:$PATH"` to `~/.zshrc`, start a new Terminal, and run `goblin++ --version` again.

4. Build from source

A source build requires Rust 1.92 or newer. From the repository root:

Terminal

```
cargo test --locked
./install.sh --prefix "$HOME/.local" --build-from-source
goblin++ doctor .
goblin++ --version
```

The lockfile pins crate versions and Cargo verifies registry checksums. A successful local build is not by itself a cross-platform release claim; the public release checklist still requires clean macOS and Linux workflow results.

5. Install the VS Code extension

- 1 Open Visual Studio Code.
- 2 Open Extensions, select the three-dot menu, and choose **Install from VSIX....**
- 3 Choose `goblinpp-vscode-0.1.7.vsix` and reload if requested.
- 4 Open a folder containing your `.gbl` files. Do not edit isolated scientific files without knowing which project owns the run ledger.
- 5 If the extension cannot find the command, set **Goblin++: Executable Path** to the absolute path of `goblin++`.

The extension supplies file recognition, syntax coloring, completions, snippets, hover help, document symbols, optional icon decorations, and explicit commands for check, run, compiled run, freeze, revise, verify, status, lineage, ledger audit, and doctor.

Authority boundary

Editor hints are assistance, not language authority. The Rust CLI decides whether syntax is valid, whether a program may run, and whether evidence verifies.

6. Create a workspace

Terminal

```
mkdir my-goblin-programs  
cd my-goblin-programs  
goblin++ doctor .
```

Goblin++ creates project custody state as needed. Keep source, run directories, and `.goblin` state together when you intend them to form one project history. Keep large research inputs under deliberate storage and backup policies.

Part III - Your first programs

7. The smallest useful program

Create `acceptance_ratio.gbl`:

```
acceptance_ratio.gbl
samples = 12
accepted = 9
fraction = accepted / samples

print("accepted fraction = {fraction:.3f}")
seal fraction
```

```
Terminal
goblin++ check acceptance_ratio.gbl
goblin++ acceptance_ratio.gbl
```

The run prints `accepted fraction = 0.750` and reports a unique `RUN_DIR`. `check` is a read-only preview: it parses the file but creates no evidence and does not consume input. The ordinary command executes through the Rust interpreter and preserves evidence.

Read the run summary

Line	Meaning
<code>RUN_STATUS=PASS</code>	The runtime completed and its policy gates passed.
<code>EXECUTION_ENGINE=rust-interpreter</code>	The default Rust evaluator ran the AST.
<code>SOURCE_SHA256</code>	Digest of the exact UTF-8 source bytes.
<code>CANONICAL_SOURCE_SHA256</code>	Digest of parsed meaning; notation aliases may share it.
<code>RECEIPT_CORE_SHA256</code>	Digest binding the core receipt fields.
<code>RUN_DIR</code>	Directory containing source, logs, receipt, seals, imports, and outputs.
<code>LEDGER_HEAD_SHA256</code>	Current checksum-ledger chain head; not authenticated authorship.

PASS means computational success

It does not mean the sample counts were collected correctly or that the ratio answers a valid scientific question.

8. Names, assignments, comments, and strings

```
language basics
# A comment continues to the end of the line.
project_name = "Night sky survey"
samples = 12
accepted = 9
complete = accepted == samples
print("{project_name}: complete = {complete}")
```

- An assignment uses `name = expression`. There is no `let` keyword.
- Names use letters, digits after the first character, underscores, and selected scientific symbols. Prefer descriptive ASCII names for portable collaboration.
- Strings use double quotes and support interpolation with `{name}`.
- Statements normally end at a newline. Braces delimit control-flow and function bodies.
- Goblin++ rejects unknown symbols rather than inventing a value.

9. Values and types

Kind	Example	Important rule
Quantity	mass = 1 kg	Stored as finite f64 SI value plus a five-part dimension vector.
Dimensionless number	samples = 12	Still a quantity, but with no physical dimension.
Boolean	ready = true	Conditions require true/false; numeric truthiness is not inferred.
Text	name = "Ada"	UTF-8 text; operations are case-sensitive and do not normalize Unicode.
Array	values = [1, 2, 3]	One-dimensional, homogeneous, copy-value, maximum 100,000 items.

There is not yet a distinct arbitrary-precision integer type. `parse_integer`, `range` arguments, indexes, and remainder are limited to exactly representable integers within $\pm(2^{53}-1)$.

10. Units and dimension checking

units.gbl

```
mass = 1 kg
distance = 2 km
time = 4 s
speed = distance / time
energy = mass * c^2

print("speed = {speed}")
print("energy = {energy}")
```

Unit	Meaning	SI factor
kg	kilogram	1
g	gram	0.001 kg
m	metre	1
km	kilometre	1000 m
s	second	1
K	kelvin	1
mol	mole	1
J	joule	kg*m^2/s^2

Goblin++ tracks mass, length, time, temperature, and amount dimensions. Addition, subtraction, and comparisons require matching dimensions. Multiplication, division, and integer powers derive new dimensions.

refused: incompatible dimensions

```
bad = 1 kg + 2 s
```

Units are semantic

A unit is not decorative text. Goblin++ refuses dimensional nonsense before it can become a plausible-looking number.

11. Registered constants

Aliases	Meaning	Status / registry
pi	Mathematical pi	Floating evaluation of exact relation
c, speed_of_light	Speed of light	Exact, SI 2019
h, planck_constant	Planck constant	Exact, SI 2019
hbar	Reduced Planck constant	Derived relation, SI 2019
G, gravitational_constant	Newtonian gravitational constant	Measured, CODATA 2022
k_B, boltzmann_constant	Boltzmann constant	Exact, SI 2019
N_A, avogadro_constant	Avogadro constant	Exact, SI 2019

A run receipt records each registered constant actually used, including its canonical identifier, source alias, SI value, dimensions, status, and registry. The future cosmological-constants registry is not part of alpha.12.

12. Operators and precedence

Family	Operators	Notes
Arithmetic	+ - * / ^	Text + text concatenates; quantity operators are dimension-aware.
Integer remainder	%	Safe dimensionless integers only; divisor cannot be zero.
Comparison	== != < <= > >=	Quantities need matching dimensions. Text/Boolean support equality only.
Boolean	not, and, or	Boolean operands only; and/or short-circuit.
Index / slice	a[i], a[start:stop]	Zero-based index; half-open copy slice.

From lower to higher binding: or, and, not, comparisons, addition/subtraction, multiplication/division/remainder, power, unary signs, and indexing. Parentheses are encouraged whenever a scientific condition could be misread.

expressions

```
eligible = quality >= 3 and not warning
odd = sample_id % 2 != 0
energy = mass * c^2
```

Part IV - Everyday language features

13. Printing and formatting

output

```
mass = 1.5 kg
fraction = 3 / 4
print("mass = {mass}")
print("fraction = {fraction:.3f}")
printf("fraction = %.3f", fraction)
```

String interpolation supports an ordinary value field and numeric `.Nf` or `.Ne` formatting. `printf` supports a deliberately small formatting subset. Standard output is preserved and hashed in every run.

14. GO_PARANOID and seal

paranoid energy program

```
GO_PARANOID

mass = 1 kg
energy = mass * c^2
print("Energy = {energy}")
seal energy
```

Feature	What it does	What it does not do
GO_PARANOID	Re-observes and preserves source bytes after execution; compares them with starting bytes; self-checks receipt before ledger registration.	It is not continuous monitoring, encryption, malware protection, or an OS sandbox.
seal name	Snapshots the current named value as a typed, hashed artifact checked by verify.	It is not required for print or generated files, and it does not encrypt the value.

Use everyday mode for learning and ordinary scripts. Use `GO_PARANOID` when the stricter source postflight evidence policy is useful. Use `seal` for values you want to collect and verify as named scientific results.

15. Conditions

branching.gbl

```
fraction = 9 / 12

if fraction >= 0.75 {
    verdict = "meets threshold"
} else if fraction >= 0.5 {
    verdict = "review"
} else {
    verdict = "below threshold"
}

switch verdict {
    case "meets threshold" { code = 1 }
    case "review" { code = 2 }
    default { code = 3 }
}

print("{verdict}; code = {code}")
```

Only the selected `if` branch runs. `switch` evaluates its selector once, then evaluates labels in order until the first exact match. It does not fall through. `default` is optional, unique, and last. Exact floating-point equality deserves care; use a tolerance condition for approximate scientific values.

16. Loops

```
loops
sum = 0
for i in range(1, 6) {
    sum = sum + i
}

for value in [1, 2, 3, 4, 5] {
    if value % 2 == 0 { continue }
    if value > 3 { break }
    print("odd value = {value}")
}

remaining = 3
while remaining > 0 {
    remaining = remaining - 1
}
```

- `range(stop)`, `range(start, stop)`, and `range(start, stop, step)` use an exclusive stop.
- A direct array loop evaluates an independent array value once.
- `break` exits the nearest loop; `continue` starts its next iteration.
- All loop bodies share a one-million-iteration limit per run, including nested loops.
- The limit catches ordinary accidental infinite loops; it is not a complete resource sandbox.

17. Arrays and slices

```
arrays.gbl
measurements = [1 kg, 2 kg, 3 kg, 4 kg]
selected = measurements[1:3]
selected[0] = 20 kg
extended = append(measurements, 5 kg)
count = len(measurements)

print("original = {measurements}")
print("slice = {selected}")
print("extended = {extended}")
```

Array elements must all be text, all Boolean, or all quantities of the same dimension. Nested arrays are not supported. Indexes are exact non-negative dimensionless integers. Slices are half-open and independent: changing `selected` does not change `measurements`. `append` returns a new copy.

18. Text and `g_strings`

```
strings
name = str_trim(" Ada pi ")
label = "Researcher: " + name
characters = len(name)
found = str_contains(name, "pi")
changed = str_replace(name, "pi", "Lovelace")
parts = str_split("red,green,blue", ",")
colors = str_join(" / ", parts)
number = parse_number("2.5")
count = parse_integer("42")

print("{label}; characters = {characters}")
print("{colors}; number = {number}; count = {count}")
```

`len(text)` counts Unicode scalar values, not bytes or grapheme clusters. Operations are case-sensitive and do not normalize Unicode. Results are bounded to 1,048,576 UTF-8 bytes; `split/join` supports at most 100,000 parts.

Conversion is explicit

Input and `argv` return text. Use `parse_number` for finite unitless decimal text or `parse_integer` for checked decimal integer syntax. Neither function parses units. Write scientific units in the Goblin++ expression.

19. Input and program arguments

interaction

```
name = input("Please enter your name: ")
print("Hello, {name}!")

print("argc = {argc}")
program = argv(0)
first_argument = argv(1)
```

Terminal

```
goblin++ greeting.gbl
goblin++ program_args.gbl -- Ada
```

Privacy boundary

Prompts, responses, and command-line arguments are preserved unredacted in hash-linked interaction evidence. Do not enter passwords, tokens, personal data, or confidential research values unless that preservation is intentional and access is controlled.

20. G funk: user-defined functions

functions.gbl

```
g_func mean_of_two(a, b) {
    total = a + b
    return total / 2
}

mean_mass = mean_of_two(2 kg, 4 kg)
print("mean mass = {mean_mass}")
seal mean_mass
```

- Declare `g_func` at top level; calls may appear before the declaration.
- Arguments are copied into a fresh local environment. Caller variables are not captured implicitly.
- Every reached path must execute `return expression`.
- A function may have at most 32 parameters; active call depth is limited to 16.
- Keep `GO_PARANOID`, `seal`, nested declarations, and inline Rust outside functions.

21. Interpretation and compilation

Terminal

```
# Default: execute with the Rust interpreter
goblin++ experiment.gbl

# Compile, execute, and preserve compiler evidence in the run
goblin++ experiment.gbl --compile

# Produce a standalone native program without executing it
goblin++ compile experiment.gbl -o experiment
./experiment
```

Compilation is explicit. Goblin++ generates reviewable Rust and calls `rustc` without a shell. Compiled runs preserve the generated source, native binary, native result manifest, stdout, stderr, and hashes.

Current compiler boundary

Alpha.12 refuses native compilation for FITS calls and generated-output calls, including calls hidden inside functions or unreachable branches. Run those programs through the default Rust interpreter. The interpreter itself is native Rust.

Part V - Scientific data and artifacts

22. Inspect a FITS file before analysis

Terminal

```
goblin++ fits-info catalogue.fits --quick
goblin++ fits-info catalogue.fits
goblin++ fits-info catalogue.fits --json
```

Quick inspection reads structural headers but deliberately skips the full file hash and is labelled as un-hashed reconnaissance. Full inspection computes SHA-256 but still does not create a run receipt. Evidence-grade import happens when a `.gbl` program executes.

Index	Convention
HDU	Zero-based: primary HDU is 0, first extension is 1.
Table row	Zero-based.
Repeated column element	Zero-based.
Image axis	One-based to match FITS NAXIS notation.

23. Read a FITS image

fits_image.gbl

```
GO_PARANOID

file = "sample.fits"
hdus = fits_hdu_count(file)
width = fits_axis(file, 0, 1)
pixels = fits_count(file, 0)
first_pixel = fits_pixel(file, 0, 0)
image_mean = fits_mean(file, 0)

print("HDUs = {hdus}; pixels = {pixels}")
print("first pixel = {first_pixel}; mean = {image_mean}")
seal image_mean
```

Supported image BITPIX values are 8, 16, 32, 64, -32, and -64. BSCALE, BZERO, and integer BLANK are honored. `fits_mean` uses finite, non-BLANK values.

24. Read a FITS binary table

fits_table.gbl

```
file = "sample.fits"
table_name = fits_header(file, 1, "EXTNAME")
rows = fits_rows(file, 1)
columns = fits_columns(file, 1)
first_object = fits_column(file, 1, "OBJECT", 0)
valid_z = fits_column_valid_count(file, 1, "Z")
mean_z = fits_column_mean(file, 1, "Z")
minimum_z = fits_column_min(file, 1, "Z")
maximum_z = fits_column_max(file, 1, "Z")

print("table = {table_name}; rows = {rows}; columns = {columns}")
print("first object = {first_object}; valid Z = {valid_z}")
print("Z range = {minimum_z} to {maximum_z}; mean = {mean_z}")
```

Direct access to a null cell produces G601. Aggregate statistics skip integer TNULL values and floating NaNs and use compensated summation. Column matching is ASCII case-insensitive. Repeated numeric/logical columns require an element index for direct reads.

25. Explicit filtered and weighted statistics

conceptual layout - keep calls on one line in alpha.12 source

```
GO_PARANOID

stats = fits_select_stats(
    "sample.fits", 1,
    "Z", 0, 3,
    "Z", "QUALITY"
)

selected_rows = stats[0]
used_rows = stats[1]
weight_sum = stats[2]
weighted_mean_z = stats[3]

print("selected = {selected_rows}; used = {used_rows}")
print("weight sum = {weight_sum}; mean Z = {weighted_mean_z}")
seal stats
```

Syntax note

Alpha.12 statements normally end at a newline. In an executable file, keep the `fits_select_stats` call on one line as shown in `examples/fits_selection.gbl`. The expanded layout above is for reading the argument groups.

The selection is lower `<= selection < upper`. Selected rows with a missing value or weight count as selected but not used. Non-missing weights must be strictly positive. The result is `[selected_rows, used_rows, weight_sum, mean]`. No catalogue cuts, masks, uncertainty model, or cosmological interpretation are inferred.

26. Write text, CSV, TSV, and JSON

conceptual layout - keep calls on one line in alpha.12 source

```
write_text(
    "summary.txt",
    "Acceptance report",
    "fraction = {fraction:.3f}"
)

write_csv("summary.csv", 2,
    "metric", "value",
    "samples", samples,
    "fraction", fraction)

write_tsv("summary.tsv", 2,
    "metric", "value",
    "samples", samples)

write_json("summary.json",
    "samples", samples,
    "fraction", fraction)
```

Generated files live only beneath `RUN_DIR/outputs`. Names must be flat ordinary filenames; absolute paths, directories, parent traversal, and duplicate declarations are refused. Each output is capped at 64 MiB. JSON preserves the SI value and dimension vector of dimensioned quantities rather than silently dropping units.

27. Create deterministic plots

conceptual layout - executable calls use one line

```
plot_fits_histogram(
    "redshift.png", file, 1, "Z",
    40, 50000, "Sampled redshift distribution")

plot_fits_scatter(
    "z_error.svg", file, 1, "Z", "Z_ERR",
    50000, "Redshift and reported error")
```

Plots use deterministic evenly spaced row indexes, skip null/NaN values, and bind their sampling metadata and input FITS hash into the receipt. SVG and lossless PNG are supported. JPEG is intentionally omitted from the authoritative path.

A plot is a view, not a selection rule

The plot title and receipt describe deterministic sampling. They do not turn sampled points into a full-population result or apply survey-specific quality cuts.

Part VI - Evidence, custody, and revision

28. What a run preserves

Evidence	Purpose
source.gbl / source metadata	Exact starting program bytes and canonical meaning.
stdout.log / stderr.log	User-visible output and failures.
receipt.json	Structured status, hashes, execution details, imports, artifacts, and policy evidence.
sealed_artifacts	Typed snapshots created by seal.
outputs	Generated text, tables, JSON, SVG, and PNG files.
data evidence	Hash-linked imported FITS bytes and access descriptions.
compiler evidence	Generated Rust, binary, and native result manifest for compiled runs.

Verification re-hashes preserved evidence. It does not re-run the scientific program or prove that the source embodied a correct hypothesis.

29. Freeze, run, verify

Terminal

```
goblin++ freeze experiment.gbl
goblin++ experiment.gbl
goblin++ verify runs/RUN_DIRECTORY
goblin++ status experiment.gbl
```

Freezing commits the exact source bytes, canonical program, constant registry, and receipt. Any later source-byte change is refused before evaluation. If an edit changes notation but not canonical meaning, Goblin++ reports that distinction, but it remains a protocol violation because exact bytes were frozen.

Freeze is strict by design

Do not overwrite history to make a frozen edit pass. Restore the bytes or create an explicit child revision.

30. Create a revision and inspect lineage

Terminal

```
goblin++ revise experiment.gbl experiment_R1.gbl --reason "correct the documented selection interval"

# Edit the child, then freeze and run it.
goblin++ freeze experiment_R1.gbl
goblin++ experiment_R1.gbl
goblin++ lineage experiment_R1.gbl
```

Revision requires a reason, refuses overwrite, copies the source into a child, and writes a lineage receipt referencing the frozen parent. The child may then be deliberately edited, frozen, and run.

31. Compare runs and audit project state

Terminal

```
goblin++ diff runs/RUN_A runs/RUN_B
goblin++ audit-ledger .
goblin++ doctor .
goblin++ capabilities
goblin++ status experiment.gbl
goblin++ lineage experiment_R1.gbl
```

Command	Question it answers
diff	Did source bytes, canonical program, sealed results, stdout/stderr, environment, or interaction differ?
audit-ledger	Does the local checksum chain verify to its recorded head?
doctor	Can this project run, write, compile, and trust its current ledger state?
capabilities	What does this exact binary claim to implement and limit?
status	Is this source unfrozen, frozen and verified, or mismatched?
lineage	Which explicit parent/child revision chain led to this source?

The diff classification is one of IDENTICAL_RESULT, NOTATION_ONLY_CHANGE, NOTATION_ONLY_REVISION, NOTATION_ONLY_CHANGE_AFTER_FREEZE, RUNTIME_DIVERGENCE, or SEMANTIC_OR_RESULT_CHANGE. Read the field-by-field comparison as well as the label; the label is a summary, not a substitute for inspection.

Checksum-only authorship

The alpha ledger detects hash-chain tampering but does not authenticate who created an event. It reports CHECKSUM_ONLY. Ed25519 signing and interrupted-head recovery remain pending.

32. Inline Rust: powerful and deliberately difficult

```
inline_rust.gbl
GO_PARANOID
x = 1

RUST_INLINE_BEGIN
println!("reviewed native operation");
RUST_INLINE_END

seal x
```

Terminal

```
goblin++ check inline_rust.gbl
goblin++ inline_rust.gbl --compile --allow-inline-rust FULL_64_CHARACTER_SHA256
```

Interpreter mode always refuses inline Rust. Compiled mode requires the exact SHA-256 of each reviewed block. A one-byte change invalidates authorization. Generated Rust and the binary are preserved and hashed.

Arbitrary native authority

Authorized inline Rust has the filesystem and network authority of the user who runs it. A content hash records exactly what was approved; it is not a sandbox. Run untrusted blocks only in a real OS sandbox or disposable environment.

Part VII - Guided day-one exercises

33. Exercise A: an acceptance report

Goal: calculate a fraction, classify it, write a small report, preserve the result, and verify the run.

conceptual source - place each function call on one line in alpha.12

```
GO_PARANOID

samples = 12
accepted = 9
fraction = accepted / samples

if fraction >= 0.75 {
    verdict = "meets threshold"
} else {
    verdict = "review"
}

print("accepted fraction = {fraction:.3f}")
print("verdict = {verdict}")
write_text("acceptance.txt",
    "accepted fraction = {fraction:.3f}",
    "verdict = {verdict}")
write_json("acceptance.json",
    "samples", samples,
    "accepted", accepted,
    "fraction", fraction,
    "verdict", verdict)
seal fraction
seal verdict
```

- 1 Save as `acceptance_report.gbl`.
- 2 Run `goblin++ check acceptance_report.gbl`.
- 3 Run `goblin++ acceptance_report.gbl` and copy the reported `RUN_DIR`.
- 4 Open `RUN_DIR/outputs` and inspect both files.
- 5 Run `goblin++ verify RUN_DIR`.
- 6 Change 9 to 8 and run again. Use `goblin++ diff RUN_A RUN_B` to see which evidence changed.

Expected reasoning

The second run should not be described merely as 'different output'. The source, canonical program, stdout, named seals, and generated files changed. The comparison lets you state exactly which layers changed.

34. Exercise B: inspect the bundled FITS fixture

Terminal

```
goblin++ fits-info examples/sample.fits --quick
goblin++ examples/fits_import.gbl
goblin++ verify examples/runs/RUN_DIRECTORY
```

- 1 Identify the primary image and the CATALOG binary-table extension.
- 2 Predict the HDU count, image pixel count, and table row count before running the example.
- 3 Read `stdout.log` and compare it with terminal output.
- 4 Inspect `receipt.json` for the imported-data digest and access descriptions.
- 5 Run `fits_selection.gbl` and explain selected rows versus used rows before interpreting the mean.

The bundled three-row FITS file is a deterministic functionality fixture, not a scientific DESI result. Its small size makes it appropriate for learning evidence flow and null/weight behavior.

35. Exercise C: compile an everyday program

everyday_alpha12.gbl

```
values = [1, 2, 3, 4, 5, 6]
sum = 0

for value in values {
    if value % 2 != 0 { continue }
    if value > 4 { break }
    sum = sum + value
}

limit = parse_integer("10")
within_limit = sum <= limit and not sum == 0
print("sum = {sum}; within limit = {within_limit}")
seal sum
seal within_limit
```

Terminal

```
goblin++ everyday_alpha12.gbl
goblin++ everyday_alpha12.gbl --compile
goblin++ diff runs/INTERPRETED_RUN runs/COMPILED_RUN
```

Both engines should print `sum = 6; within limit = true`. The run comparison should distinguish the execution engine while showing matching scientific results and seals.

Part VIII - Troubleshooting and scientific habits

36. Common errors

Code / symptom	Likely cause	First response
command not found	<code>~/local/bin</code> is not on PATH or shell has not refreshed.	Run the installed absolute path, update PATH, and start a new shell.
G002 parse failure	Malformed syntax, missing brace, unexpected token, or unsupported line break.	Run check; read the reported position; reduce to the smallest failing statement.
G101 UNKNOWN SYMBOL	Misspelled name, missing assignment, unknown function, or unsupported feature.	Check spelling and scope; do not add a placeholder value merely to silence it.
G201 incompatible dimensions	Addition/comparison between different dimensions.	Inspect units and the intended physical equation.
G202 numeric failure	Division/remainder by zero, unsafe integer, NaN/infinity, or invalid numeric text.	Validate inputs and domain assumptions.
G203 type/control failure	Wrong value kind, bad index, non-Boolean condition, loop limit, or missing function return.	Read the exact message; verify the value type and reached path.
G405 self-check failure	Paranoid evidence could not be preserved or verified.	Treat the run as failed; inspect source and evidence storage.
G501 compile failure	Unsupported compiled call, rustc problem, or generated source error.	Use interpreter for FITS/output calls; inspect preserved compiler stderr.
G601 FITS failure	Missing card/cell, unsupported structure, null direct cell, or malformed input.	Run fits-info; confirm HDU, column, row, and supported format.

37. A debugging sequence that preserves meaning

- 1 Run `goblin++ check file.gbl` before executing.
- 2 Read the complete error, including its code and stated boundary.
- 3 Confirm the input file, HDU, row, unit, and variable name rather than guessing.
- 4 Reduce the program while preserving the failing expression.
- 5 Use a new source or explicit revision when frozen work must change.
- 6 Verify failed runs too. A preserved failure can be valid evidence of refusal.
- 7 After a fix, compare old and new runs so you know which layer changed.

Do not debug by erasing history

Deleting a failed run or overwriting a frozen file may make the folder look tidy, but it destroys the very evidence needed to understand the failure. Keep what is useful; discard only deliberate disposable experiments.

38. Scientific review checklist

- State the scientific question before selecting columns, cuts, weights, or units.
- Inspect data structure and missing-value behavior before calculating aggregates.
- Label sampled plots as sampled; do not imply full-population statistics.
- Treat exact floating equality cautiously; define tolerances from the science, not convenience.
- Seal only meaningful outputs, but preserve all generated artifacts through the run system.
- Verify every run used in a report and record its RUN_DIR and receipt digest.

- Use revision reasons to explain scientific intent, not merely that a file changed.
- Validate important results independently with known values or a separate implementation.
- Keep sensitive data out of plaintext evidence unless storage and access are deliberate.
- Report unsupported semantics honestly. A refused operation is safer than an invented interpretation.

Part IX - Quick reference

39. Language keywords and directives

Word	Purpose
GO_PARANOID	Top-level opt-in source postflight evidence and receipt self-check.
seal	Snapshot a named value as a typed artifact.
g_func	Declare a top-level user function.
return	Return a value immediately from g_func.
for / in / range	Range or direct-array iteration.
while	Repeat while a Boolean condition is true.
break / continue	Exit or advance the nearest loop.
if / else	Conditional branches; else if is composed from the two words.
switch / case / default	First exact matching case, no fallthrough.
and / or / not	Short-circuit Boolean logic and negation.
true / false	Boolean literals.
argc	Read-only argument count including argv(0).
RUST_INLINE_BEGIN / END	Exact-hash-authorized arbitrary Rust block boundaries.

40. Everyday built-ins

Call	Purpose
print(values...)	Write an interpolated or rendered line to hashed stdout.
printf(format, values...)	Restricted numeric/text formatted output.
input([prompt])	Read one UTF-8 text line and preserve interaction evidence.
argv(index)	Read a zero-based text program argument.
len(value)	Array element count or text Unicode-scalar count.
append(array, value)	Return an independent array copy with one added item.
to_text(value)	Human-readable quantity/Boolean/text rendering; arrays refused.
parse_number(text)	Checked finite dimensionless decimal conversion.
parse_integer(text)	Checked safe decimal integer conversion.
str_trim(text)	Remove surrounding Unicode whitespace.
str_contains(text, needle)	Case-sensitive substring test.
str_replace(text, old, new)	Replace non-overlapping matches; old cannot be empty.
str_split(text, separator)	Split into text array; preserve empty fields.
str_join(separator, array)	Join a one-dimensional text array.

41. FITS built-ins

Call	Result
<code>fits_hdu_count(file)</code>	Number of HDUs.
<code>fits_header(file[, hdu], key)</code>	Header value.
<code>fits_axis(file[, hdu], axis)</code>	Image axis length.
<code>fits_count(file[, hdu])</code>	Image element count.
<code>fits_pixel(file[, hdu], index)</code>	Flattened image value.
<code>fits_mean(file[, hdu])</code>	Mean of finite non-BLANK image values.
<code>fits_rows(file, hdu)</code>	Binary-table row count.
<code>fits_columns(file, hdu)</code>	Binary-table column count.
<code>fits_column(file, hdu, name, row[, element])</code>	Scalar or repeated cell value.
<code>fits_column_valid_count(file, hdu, name)</code>	Non-null numeric element count.
<code>fits_column_mean/min/max(file, hdu, name)</code>	Compensated aggregate over valid numeric elements.
<code>fits_select_stats(file, hdu, selection, lower, upper, value[, weight])</code>	[selected, used, weight sum, mean].

42. Output built-ins

Call	Output
<code>write_text(name, values...)</code>	.txt or .md, one argument per line.
<code>write_csv(name, columns, cells...)</code>	RFC4180-style CSV table.
<code>write_tsv(name, columns, cells...)</code>	Tab-separated table with control separators sanitized.
<code>write_json(name, key, value, ...)</code>	JSON object with unique text keys.
<code>plot_fits_histogram(name, file, hdu, column, bins, max_rows, title)</code>	Deterministic sampled SVG or PNG histogram.
<code>plot_fits_scatter(name, file, hdu, x, y, max_rows, title)</code>	Deterministic sampled SVG or PNG scatter plot.

43. CLI command card

Command	Purpose
<code>goblin++ file.gbl</code>	Interpret and preserve a run.
<code>goblin++ file.gbl --compile</code>	Compile, execute, and preserve compiler evidence.
<code>goblin++ run file.gbl -- args</code>	Explicit run form with program arguments.
<code>goblin++ check file.gbl</code>	Read-only syntax and policy preview; no evidence.
<code>goblin++ compile file.gbl -o program</code>	Create standalone native executable.
<code>goblin++ freeze file.gbl</code>	Commit exact source and canonical meaning.
<code>goblin++ revise parent child --reason "..."</code>	Create explicit child lineage.
<code>goblin++ verify RUN_DIR</code>	Rehash and validate preserved evidence.
<code>goblin++ diff RUN_A RUN_B</code>	Classify run differences.
<code>goblin++ status file.gbl</code>	Show freeze and ledger state for source.
<code>goblin++ lineage file.gbl</code>	Show revision ancestry.
<code>goblin++ audit-ledger .</code>	Verify local checksum chain.
<code>goblin++ doctor .</code>	Check project readiness.
<code>goblin++ capabilities</code>	List implemented features and honest limits.
<code>goblin++ fits-info file.fits [--quick --json]</code>	Inspect FITS structure.

44. Current alpha.12 limits

- Numbers are f64 quantities; there is no arbitrary-precision integer type.
- Arrays are one-dimensional and homogeneous; nested collections are unsupported.
- There is no module/import system for Goblin++ source yet.
- FITS/output calls are interpreter-only, though the interpreter is native Rust.
- FITS ASCII tables, tile compression, random groups, bit/complex values, variable-length heaps, WCS interpretation, and automatic TUNIT conversion are unsupported.
- Plots are deterministic samples, not full-population statistics.
- FITS writing and bulk table export are not implemented.
- Run evidence is plaintext and not encrypted.
- The custody ledger is checksum chained but does not authenticate authorship.
- Inline Rust is arbitrary native code and needs a real sandbox when untrusted.
- Cross-platform public release reproduction and dependency-notice review remain release gates.

45. Glossary

Term	Meaning
Canonical program	Parsed semantic structure used to distinguish meaning from source notation.
Receipt	Structured record binding status, source, logs, artifacts, imports, execution, and hashes.
Seal	Explicit typed snapshot of a named value.
Freeze	Strict commitment to exact source bytes plus canonical program and registry state.
Revision	Explicit child source linked to a frozen parent and required reason.
Lineage	Parent/child history of explicit revisions.
Custody ledger	Append-only SHA-256 chain of project events; alpha authorship is not authenticated.
Notation-only change	Different source bytes with the same canonical program.
Run directory	Unique preserved execution record containing logs, receipt, evidence, and artifacts.
Evidence-first	Design principle that execution and inspectable provenance are handled together.

46. Where to go next

- 1 Run every program in `examples/`, beginning with `acceptance_ratio.gbl` and ending with `output_demo.gbl`.
- 2 Write a small program from your own field that uses one quantity, one branch, one function, and one sealed result.
- 3 If you use FITS, inspect structure first and document why each selected column, bound, and weight is scientifically appropriate.
- 4 Practice freezing a source, creating a justified revision, and comparing the two runs.
- 5 Read `docs/SECURITY.md`, `docs/PORTING_MATRIX.md`, and the public release checklist before research production use.

The Goblin rule

Trust nothing. Hash everything. Overwrite nothing. Then remember: hashes preserve what happened; data and testing reveal whether the scientific idea survives.

THE GOBLIN IS SATISFIED.

Goblin++ 0.1.0-alpha.12 | Tutorial edition 1